实验步骤

- **任务**：在模板代码基础上完成 CUDA 矩阵乘法的实现。
- **子任务**：
  - 验证 GPU 矩阵乘法结果的正确性；
  - 测试不同规模的矩阵乘法，观察实验结果。
- **目标**：
  - 熟悉 GPU 编程模型；
  - 学习 CPU 实现矩阵乘法实现思路；
  - 熟悉 CUDA 文件编译和执行过程。
- **实验代码**：`matrix_mul.cu`

0. 下载并解压实验包

从首页链接下载`lab4-5.tar.gz`，将其拷贝至用户目录，并执行命令`tar -zxvf lab4-5.tar.gz`解压实验包。

1. 打开`lab4-5`目录中的模板代码`matrix_mul.cu`，在30行处的`MatrixMulKernel`函数下实现 CUDA 矩阵乘的代码。将结果矩阵的位置索引计算代码填入指定位置（第34、38行）

```
30  __global__ void MatrixMulKernel(float* d_M, float* d_N, float* d_P, int width)
31  {
32    // Calculate the row index of the P element and M
33    // *** TO DO: Compute the row index for the current thread ***
34    // int row = ...;
35
36    // Calculate the column index of the P element and N
37    // *** TO DO: Compute the column index for the current thread ***
38    // int col = ...;
```

2. 将计算矩阵乘结果的`for`循环写入到指定位置（第46行），每个线程计算矩阵 C 一个元素的值

```
40    // Ensure the thread is within bounds
41    if ( (row < width) && (col < width) ) {
42      float pValue = 0.0;
43
44      // Each thread computes one element of the matrix
45      // *** TO DO: Implement the matrix multiplication for a single element ***
46
```

Loading [Contrib]/a11y/accessibility-menu.js

3. 将计算结果赋值给结果矩阵的对应位置（第51行）

```
48      // Store the computed value into the output matrix
49      // *** TO DO: Write the computed value to the correct position in d_P ***
50      // d_P[row * width + col] = ...;
51
```

4. 检查 `main` 函数是否调用 `MatrixMulKernel` 函数

```
206     for (int j = 0; j < nIter; j++) {
207       //matrixMulCPU(reference, h_M, h_N, m, k, n);
208       MatrixMulKernel<<<grid, block>>>(d_M, d_N, d_P, m);
209       //MatrixMulSharedMemKernel<<<grid, block>>>(d_M, d_N, d_P, m, n);
210       //cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, n, m, k, &alpha, d_N, n, d_M, k, &beta, d_P, n);
211     }
```

5. 保存文件，执行编译命令脚本，执行可执行文件 `a.out`

```
1  bash compile.sh
2  ./a.out 1 1000
```

在命令 `./a.out 1 1000` 中，“`1`”表示对矩阵乘法运算结果进行校验，“`1000`”表示测试时输入方阵的边长。

> **`compile.sh` 中的编译命令**
>
> 打开 `compile.sh` 可知脚本使用 `nvcc` 对 `matrix_mul.cu` 程序进行编译。其中，选项 `-arch=compute_35` 用于指定GPU显卡的计算能力（Compute Capability），比如 `compute_35` 表示计算能力为 `3.5`。自行安装环境的同学，需要从GPU厂商官网查询显卡对应的Compute Capability，并相应地修改 `-arch` 选项的值。

6. 查看计算GPU计算矩阵乘时间和计算结果是否正确

```
[redacted]@n1:~/lab45$ ./test0 1 1000
Kernel Elpased Time: 0.876 ms
Performance= 2282.14 GFlop/s, Time= 0.876 msec, Size= 2000000000 Ops
Computing result using host CPU...done.
Listing first 100 Differences > 0.000010...

  Total Errors = 0
```

7. 更改矩阵尺寸（矩阵为方阵）、第13行的线程块大小 `TILE_SIZE`（线程块也是方阵），对比不同参数下的计算结果

Loading [Contrib]/a11y/accessibility-menu.js

```
        @n1:~/lab45$ ./test0 0 1000
Kernel Elpased Time: 0.878 ms
Performance= 2278.98 GFlop/s, Time= 0.878 msec, Size= 2000000000 Ops
        @n1:~/lab45$ ./test0 0 3000
Kernel Elpased Time: 27.340 ms
Performance= 1975.14 GFlop/s, Time= 27.340 msec, Size= 54000000000 Ops
        @n1:~/lab45$ ./test0 0 5000
Kernel Elpased Time: 142.883 ms
Performance= 1749.68 GFlop/s, Time= 142.883 msec, Size= 250000000000 Ops
        @n1:~/lab45$ ./test0 0 10000
Kernel Elpased Time: 1722.059 ms
Performance= 1161.40 GFlop/s, Time= 1722.059 msec, Size= 2000000000000 Ops
        @n1:~/lab45$ ./test0 0 20000
Kernel Elpased Time: 16815.324 ms
Performance= 951.51 GFlop/s, Time= 16815.324 msec, Size= 16000000000000 Ops
```

```
13    const int TILE_WIDTH = 16;  // 定义块block大小
```

Loading [Contrib]/a11y/accessibility-menu.js